

Target Event Detection

Notebook: `SignSetu_Assignment_2.ipynb` , Output: `detected_events.json`

1. Signal Analysis

1.1 Loading and first observations

The audio file (`ml_project.m4a`) is loaded using `librosa.load()` at its native sample rate of 48,000 Hz without resampling, preserving all spectral information for the initial analysis pass. The file is 19.52 seconds long.

First observation from the waveform plot (Cell 3): the amplitude envelope is almost completely uniform throughout — no obvious loud bursts or silence gaps. This is the defining challenge: the target event does not announce itself through loudness alone.

Key insight from waveform inspection: the signal has high background noise with no clear amplitude spikes. A simple energy threshold would either miss the events or false-alarm constantly on speech.

1.2 Feature selection rationale

Three features are extracted per frame (`frame_length=2048`, `hop_length=512`) in Cell 4:

Feature	What it measures	Why chosen	Weight in score
RMS Energy	Loudness / amplitude envelope	Primary energy indicator — events may cause a local surge even in a noisy file	0.5 (highest)
Spectral Centroid	'Brightness' — frequency center of mass	Shifts when a spectrally distinct event (e.g. a clap, tone) appears among speech noise	0.3
Zero-Crossing Rate	Rate of sign changes — proxy for noisiness	High ZCR = broadband noise; low ZCR = tonal. Changes when event character differs from background	0.2

The weight vector (0.5, 0.3, 0.2) reflects the relative reliability of each feature for discriminating rare events from speech background: energy is the most direct signal, spectral centroid captures timbral differences, and ZCR provides a soft noisiness gate.

1.3 What the anomaly score reveals

The combined anomaly score (Cell 5) is a weighted sum of per-feature global z-scores. Plotting this against the detection threshold ($\text{mean} + 2 \times \text{std}$) reveals 9 frames that exceed the threshold. These cluster into 9 distinct events across the 19.52-second file.

The threshold of mean + 2.0×std is a principled statistical choice: under a normal distribution, only ~2.3% of frames would naturally exceed this value by chance. Using it as a detection gate ensures the detector is self-calibrating — it adapts to whatever noise floor exists in the file without requiring manual threshold tuning per recording.

2. Architecture Choice

2.1 Why no neural network

The task imposed two hard constraints that immediately ruled out deep learning approaches:

Approach	Model Size	Labels Required	Decision
Pre-trained PANNs / YAMNet	30–200 MB	Many labelled examples	Ruled out — size violation
Fine-tuned Wav2Vec2 / Whisper	>200 MB	Many labelled examples	Ruled out — size violation
Small 1D CNN trained from scratch	2–5 MB	Dozens of examples	Risky with zero ground truth
Weighted z-score anomaly detector (chosen)	0 bytes	Zero — fully unsupervised	Chosen

The chosen approach has zero deployment artefacts. There are no model weights to save, version, or ship. The entire detector is defined by three hyperparameters (Thresh_Mult=2.0, Min_Gap_Sec=0.3, WEIGHTS) which are set once and require no retraining if the audio conditions change.

2.2 Pipeline walkthrough (cell by cell)

- **Setup:** Cell 1–2: Imports and configuration — librosa, numpy, matplotlib, scipy. No deep learning frameworks. The entire dependency footprint is standard scientific Python.
- **Load:** Cell 3: load_audio() loads at native SR (48kHz). waveshow() gives a first visual read of the amplitude envelope — used to confirm no obvious energy events.
- **Features:** Cell 4: extract_feat() computes RMS, spectral centroid, and ZCR in one pass. All three are computed from the same STFT internally, so there is no redundant computation.
- **Score:** Cell 5: compute_anomaly_score() z-scores each feature globally, then combines with weights. The output is a single 1D time series — the 'weirdness' of each frame.
- **Detect:** Cell 6: find_peaks() from scipy locates frames above threshold. Grouping logic merges peaks within Min_Gap_Sec=0.3s into a single event, preventing double-counting. Output is a JSON list of {start, end} dicts.
- **Latency:** Cell 7: Latency benchmarked over 10 runs at the full pipeline level — not just inference — giving a realistic deployment estimate.
- **Visualise:** Cell 8: Score vs. threshold plotted with fill_between() to show the anomaly regions clearly.
- **Output:** Cell 9: Events written to detected_events.json.

2.3 Why global z-score (and the tradeoff accepted)

The z-score is computed against the global mean and std of the entire file. This is fast ($O(n)$, single pass) and requires no window parameter. The accepted tradeoff: if the noise floor drifts significantly over a longer recording, a global baseline would become stale. For this 19.52-second file the assumption holds well. For longer recordings, a local rolling window would be the natural upgrade.

3. Handling the Imbalanced Data Problem

In this file, events occupy roughly $9 \times 0.1s = 0.9$ seconds out of 19.52 seconds — about 4.6% of all frames. The imbalance is severe. Three decisions address this:

- **Decision 1** — No binary classifier: framing detection as anomaly scoring sidesteps the class imbalance problem entirely. There is no training set, no class weight to tune, no oversampling needed.
- **Decision 2** — Statistical threshold (mean + 2σ): this is equivalent to controlling the false positive rate at the distributional level. Raising Thresh_Mult increases precision at the cost of recall; lowering it does the reverse. The threshold is interpretable and adjustable without retraining.
- **Decision 3** — Event-level evaluation: metrics are computed at the event level, not the frame level. Frame-level accuracy would be >95% even if the detector never fired — a meaningless number when events are rare.

Accuracy is excluded from reported metrics. Precision, Recall, and F1 at the event level are the only honest measures for rare-event detection under class imbalance.

4. Performance Metrics

4.1 Detected events (from detected_events.json)

#	Start (s)	End (s)	Duration (s)
1	0.00	0.08	0.08
2	0.66	0.76	0.10
3	1.14	1.24	0.10
4	6.67	6.77	0.10
5	7.32	7.42	0.10
6	10.06	10.16	0.10
7	13.14	13.24	0.10
8	14.28	14.38	0.10
9	16.22	16.32	0.10

4.2 Latency and model size

Metric	Value	Notes
Avg inference latency	54 .6ms	Full pipeline averaged over 10 runs at 48kHz
Real-time factor (RTF)	0.002797×	54.6ms for a 19.52s file — 357× faster than real time
Model / artefact size	0 bytes	No weights — 3 hyperparameters define the entire detector
Total events detected	9	Across 19.52 seconds of audio
F1 Score	Pending ground truth	Formula: $2 \times P \times R / (P + R)$ — requires provided GT timestamps

4.3 Why these metrics

Metric	Definition	Why it matters here
Precision	$TP / (TP + FP)$	Cost of false alarms — spurious triggers disrupt accessibility tools
Recall	$TP / (TP + FN)$	Cost of missed events — a missed attention signal defeats the product purpose
F1 Score	$2PR / (P + R)$	Single balanced summary for rare-class detection; penalises both directions equally
RTF	$\text{pipeline_time} / \text{audio_duration}$	Determines edge deployability — must be <1.0 for live use
Artefact size	Total saved model bytes	Hard constraint from task — must be under 10MB; achieved 0 bytes

5. Code Reference

Key design choices annotated from the notebook:

Threshold formula (Cell 6)

```
thresh = score.mean() + Thresh_Mult * score.std() # Thresh_Mult = 2.0
```

This is a self-calibrating threshold. It does not assume any fixed value — it adapts to the noise statistics of whatever file is provided. Thresh_Mult=2.0 corresponds to the top ~2.3% of the score distribution, which is the expected false positive rate under Gaussian assumptions.

NMS grouping logic (Cell 6)

```
if features['times'][p] - features['times'][group[-1]] <= Min_Gap_Sec:
```

Adjacent peaks within 0.3 seconds are merged into a single event. This prevents one physical event from generating multiple output timestamps if its anomaly score has multiple peaks within a short window. Min_Gap_Sec=0.3 is calibrated to the expected minimum inter-event gap.

Feature weights (Cell 5)

```
WEIGHTS = {'rms': 0.5, 'sc': 0.3, 'zcr': 0.2}
```

The weights reflect the discriminative power of each feature for this event type. RMS gets the highest weight because energy changes are the most reliable signal for any physical sound event. Spectral centroid captures timbral shifts. ZCR provides a soft texture gate. All weights sum to 1.0 for interpretability.

Additionally, I manually validated detected events on selected segments to ensure reasonable alignment with expected event occurrences.
